WARNING Where possible, I recommend that you don't directly inspect paths in your middleware like this. A better approach is to use endpoint routing instead, as I discussed in chapter 31. The middleware in this section is for demonstration purposes only.

Middleware is slightly complicated to unit-test because the `HttpContext` object is conceptually a big class. It contains all the details for the request and the response, which can mean there's a lot of surface area for your middleware to interact with. For that reason, I find unit tests tend to be tightly coupled to the middleware implementation, which is generally undesirable.

For the first test, you'll look at the case where the incoming request `Path` doesn't start with `/ping`. In this case, `StatusMiddleware` should leave the `HttpContext` unchanged and call the `RequestDelegate` provided in the constructor, which represents the next middleware in the pipeline.

You could test this behavior in several ways, but in listing 36.2 you test that the `RequestDelegate` (essentially a one-parameter function) is executed by setting a local variable to `true`. In the `Assert` at the end of the method, you verify that the variable was set and therefore that the delegate was invoked. To invoke `StatusMiddleware`, create and pass in a `DefaultHttpContext`, which is an implementation of `HttpContext`.

NOTE The `DefaultHttpContext` derives from `HttpContext` and is part of the base ASP.NET Core framework abstractions. If you're so inclined, you can explore the source code for it on GitHub at http://mng.bz/MB9Q.